



Article

Hierarchical Runtime Anomaly Diagnosis from Real PX4 Telemetry with Commit-Level Architecture-Aware Inspection Evidence

Jiao Wu ^{1,2,†}, Bo Liu ^{1,†}, Yilan Li ³, Chenglie Du ¹ and Junsheng Wu ^{1,*}

¹ Northwestern Polytechnical University, Xi'an 710072, Shaanxi, China; wjiao@mail.nwpu.edu.cn

² Xi'an Aeronautics Computing Technique Research Institute, Aviation Industry Corporation of China, Ltd., Xi'an 710065, Shaanxi, China;

³ Xi'an University of Technology, Xi'an, Shaanxi, China

* Correspondence: wujunsheng@mail.nwpu.edu.cn

† These authors contributed equally to this work.

Highlights

- A two-stage hierarchy separates real-flight anomaly detection from four-domain diagnosis.
- TreeSHAP evidence is conserved across features, channels, uORB topics, and subsystems.
- Commit-matched, Stage-1-gated analysis exposes detector losses before module inspection.
- Strict isolation and controlled replay expose leakage sensitivity and detector-gated limits.

Abstract

Runtime anomaly detection identifies abnormal UAV telemetry but does not directly provide fault-domain or software inspection evidence. HS-AeroTS combines a Light-GBM detector, a conditional four-domain classifier, hierarchical TreeSHAP aggregation, and a static PX4 uORB source-tree graph. On 1,389 UAV-SEAD logs, five-seed fixed-chronological AUPRC was 0.7522 ± 0.0039 (95% flight-log CI [0.6915, 0.8080]); anomaly-only fault-domain Macro-F1 was 0.8980 ± 0.0020 [0.8581, 0.9296], and complete Cascade Macro-F1 was 0.6902 ± 0.0071 [0.6308, 0.7350]. Under leave-log-out isolation, detector AUPRC decreased to 0.6296 ± 0.0023 [0.4945, 0.7456]. Hierarchical aggregation conserved evidence mass, and attribution-guided masking caused greater degradation than random masking, but a document-derived semantic-map sensitivity showed near-random External Position and below-random Global Position Consistency@1. The ground-truth-anomaly-conditioned architecture analysis contained 1,393 matching-commit anomalous windows. Applying the frozen Stage 1 gate retained an average of 937.2 of these windows, missed 455.8, and introduced 532.2 matching-commit false-positive windows. Conservative source-path exclusion reduced the graph from 180 edges and 54 modules to 145 edges and 36 modules while retaining all 18 topics; this is a source-tree sensitivity, not build-target validation. In controlled replay, onset-conditioned ranking produced nonzero module recall with wide confidence intervals, but the unchanged detector identified 0 of 12 mutation runs. HS-AeroTS therefore provides conditional, traceable evidence for engineering inspection rather than causal or deployment-validated root-cause localization.

Keywords: UAV telemetry; runtime anomaly detection; fault-domain diagnosis; explainable machine learning; PX4; uORB; source-tree inspection candidates

Received:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Drones* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\) license](#).

1. Introduction

1.1. Engineering Problem

Reliable autonomous flight requires more than recognizing that a multivariate telemetry sequence has departed from normal operation. A binary anomaly decision is valuable for surveillance and safety monitoring, and real-time methods have demonstrated that abnormal UAV behavior can be detected from onboard flight data [1,2]. In a flight-control engineering workflow, however, the same alert must also support a decision about what to inspect. The engineer needs to know which fault domain is implicated, which telemetry channels provide the strongest evidence, and which parts of the flight-control software consume or produce the corresponding signals. This distinction reflects the broader separation between fault detection and fault diagnosis in UAV systems [3–5].

The distinction is especially important for heterogeneous PX4 logs. Position estimates, estimator innovations, actuator responses, inertial measurements, battery states, and system-health variables are coupled through the closed-loop vehicle dynamics. Consequently, a high anomaly score alone cannot distinguish an external-position disturbance from a global-position, altitude, or mechanical/electrical anomaly. These domains imply different follow-up checks even when their telemetry deviations overlap. The UAV-SEAD corpus provides real PX4 flight logs with these runtime anomaly categories [6]; our data audit retained 1,389 usable logs, establishing a sufficiently broad but class-imbalanced basis for evaluating both binary detection and subsequent fault-domain diagnosis.

Actionable diagnosis also requires an interpretable path from model output to the flight-control architecture. PX4 is organized as a set of modules that exchange typed messages through its publish–subscribe middleware [7,8]. A telemetry channel can therefore be associated with a logged uORB topic and a functional subsystem, while a publisher/subscriber graph reconstructed for a specified firmware revision can identify software modules connected to that topic. Such a chain turns statistical evidence into architecture-aware inspection leads. It does not, by itself, prove that any connected module caused the observed behavior; the resulting modules are suspects to prioritize during engineering inspection rather than localized source-code faults.

This evidentiary boundary is central to the problem formulation. Labels attached to real flights describe observed runtime conditions or fault domains, not defective PX4 statements, functions, or modules. Treating those labels as source-code ground truth would collapse two different inference tasks: diagnosing vehicle behavior from telemetry and localizing an injected or known software defect. We therefore formulate the engineering problem as a hierarchy: first detect abnormal telemetry, then diagnose the affected fault domain, expose the telemetry evidence supporting that decision, and finally use PX4 architecture only to rank plausible module inspection leads. Evidence for the first two stages is provided by the audited real-flight corpus and the detection and fault-domain experiments, whereas claims about software-module ranking require separate controlled evidence.

1.2. Research Gap and Proposed Approach

The unresolved research gap is the absence of a jointly evaluated path from a telemetry alarm to an engineering inspection lead. UAV anomaly-management and fault-diagnosis studies primarily evaluate whether abnormal behavior can be detected or assigned to a fault category [2,3,5]. Explainable machine-learning studies instead evaluate how model predictions can be attributed to input features [9,10], while software fault-localization studies

rank code entities using program-level evidence [12]. These lines address different parts of the diagnostic process, but their outputs and ground truths are not directly interchangeable. Evaluating them separately therefore leaves unanswered whether one abnormal flight can be followed consistently from detection, through fault-domain diagnosis and telemetry evidence, to a defensible set of PX4 module inspection leads.

A second gap concerns traceability across representation levels. Temporal-statistical models operate on descriptors such as windowed means, extrema, variations, or slopes, whereas a flight-control engineer reasons about physical telemetry channels, logged topics, and functional subsystems. Feature attributions can quantify the contribution of individual model inputs, but they do not inherently recover this engineering hierarchy. Moreover, a PX4 module relationship cannot be inferred from a feature name alone: it depends on the firmware version and on the uORB topics published or subscribed by each module [7,8]. Without an explicit and quantitatively conserved mapping from statistical feature to telemetry channel, topic, subsystem, and module, feature importance remains difficult to translate into reproducible inspection priorities.

A third gap is the validation of software-module rankings. Static topic dependencies show architectural connectivity, not execution, defect, or causality. Software fault-localization research evaluates suspiciousness rankings against known faulty entities, and prior work has shown that conclusions can depend strongly on the faults and evaluation protocol used [12,13]. Controlled source mutations can provide explicit module ground truth, although mutation evidence must itself be interpreted with care because artificial and real faults are not equivalent [14]. For a telemetry-driven cascade, validation must additionally preserve the detection gate: an onset-conditioned ranking may reveal useful downstream evidence when an anomalous interval is supplied, but it does not constitute an end-to-end localization result if the runtime detector never activates.

To address these gaps, this paper develops HS-AeroTS as a hierarchical diagnostic evidence framework. The earlier “FL” suffix is removed because the evaluated system does not establish fault localization. A first-stage detector scores telemetry windows as normal or anomalous, and a second-stage classifier assigns detected anomalies to External Position, Global Position, Altitude, or Mechanical/Electrical domains. TreeSHAP evidence is then aggregated through an explicit feature–channel–topic–subsystem hierarchy, after which a static PX4 uORB graph reconstructed at the majority firmware commit propagates topic evidence from matching-commit windows to source-tree inspection candidates. The framework keeps each evidence level separately testable: detection and fault-domain diagnosis are evaluated on real-flight labels, explanation quality is assessed through conservation, masking, and semantic-mapping sensitivity, and module ranking is examined with controlled mutation replay under both onset-conditioned and detector-gated protocols. Ground-truth-anomaly-conditioned, Stage-1-gated, and detector-gated results are kept distinct so that architecture-aware associations are not presented as source-code root causes or deployment performance.

1.3. Research Questions and Contributions

The study is organized around three research questions that progress from runtime behavior to architecture-aware diagnostic evidence:

- RQ1.** How accurately and reproducibly can real PX4 telemetry support runtime anomaly detection and four-class fault-domain diagnosis under fixed chronological, purged, leave-log-out, and external-data protocols?
- RQ2.** To what extent does hierarchical SHAP aggregation provide conserved, prediction-relevant under masking, and label-consistent evidence from temporal-statistical features to telemetry channels, uORB topics, and functional subsystems?

RQ3. To what extent can uORB dependencies reconstructed at a specified PX4 commit convert topic evidence into useful software-module suspect rankings under controlled source mutations, and how do firmware-version coverage and the runtime detection gate constrain the end-to-end capability?

Figure 1 summarizes the model chain, the SHAP/uORB evidence chain, and the three validation branches. Table 1 links each research question to its experiments, primary metrics, and permitted evidence level. This structure deliberately separates real-flight runtime labels, explanation validation, static architecture associations, and source-mutation ground truth so that results at one level are not used as unsupported evidence at another.

The work makes three contributions:

1. A reproducible protocol for runtime anomaly detection and four-class fault-domain diagnosis on real PX4 telemetry. The protocol fixes data provenance, log-level isolation, feature construction, model configuration, and imbalance-aware metrics, and it evaluates sensitivity under stricter purged and leave-log-out splits as well as external and weak-label checks.
2. A traceable SHAP/uORB evidence chain that aggregates temporal-statistical feature contributions to telemetry channels, topics, and subsystems before using PX4 dependencies reconstructed at the majority firmware commit to produce software-module suspects. Conservation, feature masking, and label-consistency tests make the intermediate evidence quantitatively auditable while retaining their non-causal scope.
3. A boundary-aware evaluation that distinguishes conditional downstream ranking from detector-gated end-to-end behavior. Strict data splits, external telemetry, and controlled source-mutation replay quantify where the diagnostic evidence remains informative and where module-level claims are not supported.

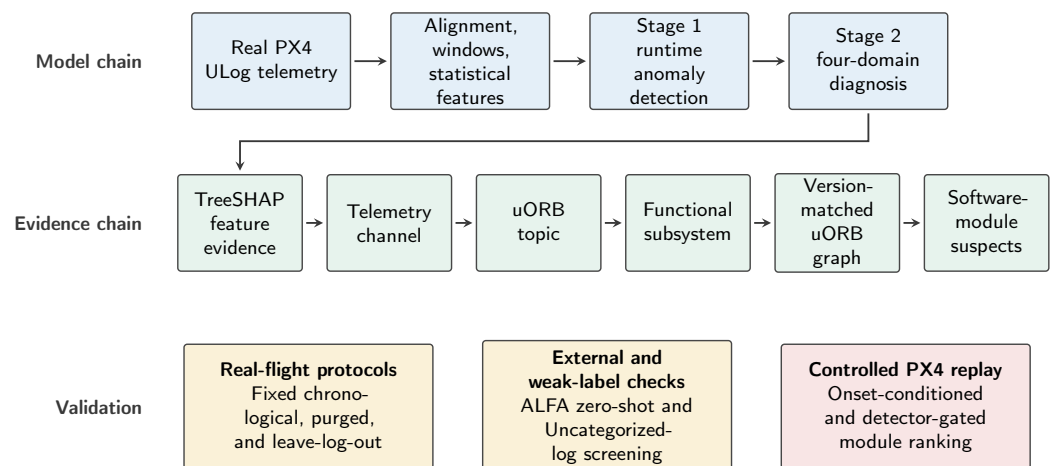


Figure 1. Overview of HS-AeroTS. The model chain performs runtime anomaly detection followed by four-domain diagnosis. TreeSHAP evidence is aggregated from statistical features to telemetry channels, uORB topics, and subsystems, and a static uORB graph reconstructed at the selected majority firmware commit produces software-module suspects. Three validation branches separately test real-flight robustness, external or weak-label transfer, and controlled replay under onset-conditioned and detector-gated protocols.

Table 1. Research questions, corresponding experiments, primary metrics, and evidence levels.

RQ	Corresponding experiments	Primary metrics	Evidence level
RQ1	Runtime anomaly detection and four-domain diagnosis under fixed chronological, purged, leave-log-out, ALFA zero-shot, and Uncategorized-log protocols	AUPRC, AUROC, Macro-F1, Balanced Accuracy, per-class Recall	Runtime anomaly and fault-domain evidence; no source-code ground truth
RQ2	SHAP conservation, top- <i>k</i> masking versus random masking, Consistency@ <i>K</i> /Hit@ <i>K</i> , and majority-commit topic-module mapping	Conservation error, Macro-F1 decrease, Consistency@ <i>K</i> , Hit@ <i>K</i> , mapping coverage and firmware-version coverage	Feature, channel, topic, subsystem, and static architecture-association evidence
RQ3	Controlled PX4 ULog replay with known source mutations; onset-conditioned and detector-gated evaluation	Detector recall, Top-1/3/5, MRR, EXAM	Mutation-ground-truth ranking evidence, explicitly conditioned on the evaluation gate

2. Related Work

2.1. UAV Telemetry Anomaly Detection and Fault Diagnosis

UAV telemetry fault detection and diagnosis combines system-specific flight knowledge with multivariate time-series analysis. Surveys distinguish model-based, signal-based, knowledge-based, and data-driven approaches, while also noting that real-flight evidence is less abundant than simulation or laboratory validation [3,4,15]. Flight logs record coupled state estimates, estimator innovations, inertial measurements, actuator responses, power variables, and system-health indicators; an abnormal condition may therefore appear across several channels rather than as an isolated sensor excursion. Keipour et al. developed an online detector for autonomous aerial vehicles and released ALFA as a real-flight fault and anomaly dataset [1,16]. More recently, UAV-SEAD organized real PX4 logs around state-estimation-related runtime anomalies in External Position, Global Position, Altitude, and Mechanical/Electrical domains [6]. These datasets make windowed multi-source telemetry a practical basis for both anomaly detection and fault-domain diagnosis, but their labels, channel availability, and operating conditions remain dataset-specific.

General multivariate time-series anomaly detection provides several complementary modeling paradigms for such data. Isolation Forest detects sparse deviations through random partitioning without explicitly modeling temporal order [17]. Reconstruction and probabilistic sequence models instead learn normal temporal or inter-channel structure, as illustrated by MSCRED and OmniAnomaly [18,19]. Graph-based methods such as GDN learn dependencies among sensors, whereas TranAD and Anomaly Transformer use attention to represent longer temporal relationships [20–22]. Recent methods further model cross-channel structure through frequency-domain patching in CATCH or a Granger-causality perspective in GCAD [23,24]. These methods broaden the available baselines for multi-source flight data, but their primary output is still an anomaly score or binary decision. They do not inherently map a detected window to the UAV-specific fault domains used for engineering diagnosis, nor do they establish a traceable relationship between an influential signal and the flight-control software architecture.

For real UAV telemetry, AeroTSBoost is the closest detector-level reference because it represents fixed-length flight windows with temporal-statistical descriptors and applies gradient-boosted trees to anomaly mining [2]. Its use of compact descriptors and LightGBM [25] provides a reproducible alternative to sequence models when the channel set is heterogeneous and the data are strongly imbalanced. The present study therefore uses an AeroTSBoost-compatible Stage 1 protocol for binary runtime anomaly detection, while

treating binary detection as only the entry point to diagnosis. Stage 2 conditionally assigns detected anomalous windows to one of the four UAV-SEAD fault domains, and a Direct Five-Class model is retained as a non-cascaded comparator. This design separates the question of whether a window is abnormal from the question of which fault domain best explains the labeled runtime condition.

Evaluation choices are particularly consequential for imbalanced, overlapping telemetry windows. Accuracy and AUROC can remain high when normal windows dominate, whereas precision–recall analysis exposes the trade-off on the minority anomaly class [26,27]. Accordingly, AUPRC is emphasized for binary detection, while Macro-F1, Balanced Accuracy, and per-class Recall are used for fault-domain diagnosis so that minority domains cannot be hidden by aggregate accuracy. Time-series evaluation must also account for temporal range labels and partition dependence [28,29]. Randomly assigning overlapping windows can place nearly identical samples, or samples from the same flight log, in both training and test sets; broader benchmark analyses have shown that such evaluation choices can create optimistic conclusions [30]. The experiments in this study therefore report fixed chronological results together with purged and leave-log-out protocols, and treat ALFA only as a zero-shot binary reference because its label space does not support direct four-domain transfer.

2.2. Explainable Time-Series Models

Post-hoc attribution methods explain a model prediction by assigning relevance to its inputs, but their mechanisms and units of explanation differ. LIME fits a local surrogate around an individual prediction, Integrated Gradients accumulates input gradients along a path from a reference, and SHAP represents a prediction through additive feature contributions grounded in Shapley values [9,31,32]. For sequential models, TimeSHAP adapts perturbation-based Shapley analysis to recurrent time-series predictions and can attribute relevance to features, events, and time steps [33]. These approaches make otherwise opaque predictions more inspectable, yet their immediate outputs remain tied to the model input representation. In windowed UAV telemetry, an attribution to a statistical descriptor is not automatically an explanation at the physical-channel or fault-domain level.

Tree ensembles provide a useful setting for structured attribution because TreeSHAP computes additive contributions for tree-model outputs and supports aggregation from local explanations to broader patterns [9,10]. When each temporal-statistical feature has a fixed provenance, contributions can be grouped from descriptor to telemetry channel and then to a logged topic or functional subsystem. Hierarchical aggregation improves engineering readability, but it also introduces a bookkeeping requirement: every lower-level feature must have a defined parent, and the total evidence magnitude should be preserved when groups form a disjoint partition. A visually plausible subsystem ranking is therefore insufficient unless the aggregation can be traced back to the original model features and its numerical conservation is checked explicitly.

Explanation evaluation must also test whether attributed features are functionally important to the prediction. Sanity-check research has shown that visually appealing saliency patterns can remain insensitive to model parameters or data relationships [34]. Removal-based benchmarks instead evaluate faithfulness by perturbing highly ranked inputs and measuring the resulting performance change [35]. Broader evaluation frameworks, including Quantus, organize such tests into complementary properties rather than treating one score as a complete validation [36]. For time-series diagnosis, top-*k* masking is particularly useful when it is compared with an equal-size random mask: a larger performance decrease under attribution-guided masking provides evidence that the selected features are prediction-relevant under the chosen perturbation, while the random baseline controls for

the generic effect of removing inputs. Median replacement can create off-manifold samples and therefore does not prove feature necessity in an interventionist or causal sense.

Faithfulness to the predictor does not by itself establish consistency with a diagnostic label. In a structured UAV setting, the highest-ranked channels can additionally be compared with a predeclared semantic channel-to-subsystem mapping through Consistency@K or Hit@K. This test asks whether the explanation emphasizes telemetry heuristically associated with the labeled fault domain, whereas masking asks whether the same evidence is prediction-relevant under perturbation; the two criteria are related but not interchangeable. The present framework therefore evaluates explanations at three levels: numerical conservation of hierarchical aggregation, perturbation sensitivity through attribution-guided versus random masking, and agreement with a predefined semantic mapping through top- k channel matching. These tests support the interpretation of SHAP values as organized diagnostic evidence, but they do not convert feature attribution into physical causality, independent ground truth, or root-cause localization.

2.3. PX4 Architecture-Aware Diagnostic Evidence

PX4 organizes flight-control functionality as interacting modules rather than as a monolithic program. Its middleware, uORB, provides typed publish-subscribe communication through which estimators, controllers, sensor drivers, and other services exchange state and command data [7,8]. The same topic vocabulary is represented in ULog telemetry, creating a practical bridge between an influential logged channel and the software components that publish or consume its corresponding uORB topic. This bridge is useful only when firmware provenance is retained: module boundaries, topic definitions, and communication relations can change across PX4 revisions, so an architecture map built for a different version may produce misleading suspects.

The uORB graph supports several complementary interpretations of diagnostic evidence. A producer-oriented view emphasizes modules that may have generated an abnormal value; a consumer-oriented view emphasizes modules whose behavior may have been affected by that value; and a bidirectional view retains both roles when the available evidence does not justify choosing between them. Propagating topic-level importance over these relations can therefore reduce a large source tree to a reviewable set of software-module suspects. Nevertheless, this evidence is structural and version-specific. Subscription does not establish that the relevant execution path was active, publication does not establish that the produced value was erroneous, and high-connectivity topics may implicate many modules. Architecture can narrow the inspection surface, but connectivity alone cannot identify a cause.

This setting differs from conventional software fault localization. Spectrum-based approaches rank statements, methods, or files by contrasting their execution in passing and failing tests; early visualization work demonstrated how such suspiciousness information can guide inspection, and subsequent surveys systematized a broad family of statistical ranking techniques [11,12]. Their evaluation normally assumes a known faulty entity and reports measures such as Top- k accuracy, mean reciprocal rank, or inspection effort. Studies using seeded faults and real defects have also shown that conclusions can depend strongly on the benchmark, fault population, and evaluation protocol [13,14]. Real-flight telemetry generally provides neither program-coverage spectra nor verified source-code defect labels, so applying the causal language or ground-truth assumptions of test-based fault localization would be unwarranted.

Accordingly, the present work positions commit-specific topic-to-module propagation as an architecture-aware diagnostic evidence layer, not as causal root-cause analysis. Runtime anomaly labels support evaluation of anomaly detection and fault-domain di-

agnosis, whereas controlled replay with known source mutations is required to evaluate software-module suspect ranking against a source-level target. Even in that setting, onset-conditioned ranking must be distinguished from detector-gated end-to-end ranking: if the runtime detector does not flag a run, the downstream localization chain produces no result. The resulting module list should thus be read as prioritized evidence for engineering inspection, conditional on the observed telemetry, the explanation model, and the reconstructed PX4 revision; flights from other revisions require their own mapping or an explicit compatibility analysis.

2.4. Positioning Summary

The preceding literature addresses complementary parts of the diagnostic problem, but usually under different task definitions and evidence assumptions. UAV fault-detection studies and multivariate time-series detectors primarily determine whether telemetry is abnormal, while AeroTSBoost provides the closest real-flight, temporal-statistical detector reference for the present setting [1,2]. Explainability research supplies attribution and faithfulness tests, but its outputs remain tied to model inputs unless an explicit aggregation and architecture mapping is defined [10,35]. Conversely, spectrum-based software fault localization ranks program entities from test executions and known pass/fail outcomes, evidence that routine real-flight logs do not provide [12]. These research lines are therefore individually relevant, yet none of their typical formulations spans the complete path from a detected telemetry window to validated diagnostic evidence and version-aware PX4 software-module suspects.

HS-AeroTS is positioned as an auditable integration of these evidence levels rather than as a replacement for their underlying methods. It couples real-flight runtime anomaly detection with four-domain diagnosis, evaluates the resulting hierarchical SHAP evidence through conservation, masking, and label-consistency tests, and propagates topic evidence over a uORB graph reconstructed at the selected majority firmware commit. Its evaluation also reports fixed chronological, purged, and leave-log-out protocols, together with external transfer and controlled replay. Crucially, conditional module-ranking results are reported alongside the detector-gated replay failure, preventing onset-conditioned success from being interpreted as reliable end-to-end localization. Table 2 summarizes this distinction; the entries describe the typical scope of each research line, not comparative predictive performance.

Table 2. Positioning of representative research lines and HS-AeroTS across the four evidence dimensions emphasized in this study. “Task-dependent” indicates that the method can be attached to different predictive tasks but does not itself define the listed diagnosis level.

Research line	Runtime anomaly detection	Fault-domain diagnosis	Explanation validation	Software-module evidence
UAV fault detection and diagnosis [3,4,16]	Flight- or signal-level detection	Fault or failure categories	Not a primary evaluation target	No source-architecture mapping
General multivariate time-series anomaly detection [19–21]	Anomaly scores or labels	Not intrinsic	Usually outside detector evaluation	None
AeroTSBoost [2]	Real-flight binary detection	Not addressed	Not addressed	None
Attribution and explanation evaluation [33–35]	Task-dependent	Task-dependent	Attribution sanity or perturbation tests	No PX4 architecture mapping
Spectrum-based software fault localization [11–13]	No telemetry detector	No physical fault domain	Ranking against known faulty entities	Execution-spectrum entity ranking
HS-AeroTS	Real-flight binary detection	Four UAV-SEAD domains	Conservation, masking, and semantic-map agreement	Matching-commit uORB module suspects

3. Materials and Methods

3.1. Data Preprocessing and Feature Representation

UAV-SEAD was used as the primary real-flight dataset [6]. To make the data inventory reproducible, the dataset revision was fixed to 774a336a1d1d06b248927a35c2ea1f6869e9e1a4, and the original PX4 ULog files were retained as read-only inputs. Inventory and parsing checks excluded one missing file, two parse failures, and four excessive-duration or corrupt logs, leaving 1,389 usable logs. Numeric fields from the mapped PX4 topics were then screened by log-level availability. A channel was retained when it occurred in at least 60% of the usable logs, producing a common set of 87 telemetry channels; all aligned arrays and feature matrices were generated as derived artifacts by versioned scripts and configuration files.

Because uORB topics are logged asynchronously, each flight was resampled independently onto a uniform 10 Hz grid. Finite samples within a channel were linearly interpolated by timestamp, values outside the observed interval were extended from the nearest finite endpoint, and a channel absent from an entire log was filled with zero. Flight boundaries were never crossed during interpolation. This procedure yielded 1,892,063 aligned records across the 1,389 logs. The log identifier and the original annotation intervals were retained alongside every aligned sequence so that subsequent partitioning and window labeling could remain flight-aware.

Each aligned sequence was converted into overlapping input windows of $L = 96$ samples with stride $s = 8$. A horizon of $h = 12$ additional samples was used for label assignment but was not included in the input representation: a binary window label was positive if an annotated anomaly occurred anywhere from the window start through the end of this horizon, and the fault-domain label was the majority nonzero annotated class over the same interval. Under the primary chronological protocol, this construction produced 218,537 windows. Split assignment preceded normalization. For channel c , the

mean μ_c^{tr} and standard deviation σ_c^{tr} were estimated only from samples participating in training windows, after which all partitions were transformed as

$$z_{c,t} = \frac{x_{c,t} - \mu_c^{\text{tr}}}{\tilde{\sigma}_c^{\text{tr}}}, \quad \tilde{\sigma}_c^{\text{tr}} = \begin{cases} \sigma_c^{\text{tr}}, & \sigma_c^{\text{tr}} \geq 10^{-6}, \\ 1, & \sigma_c^{\text{tr}} < 10^{-6}. \end{cases} \quad (1)$$

This ordering prevents validation and test telemetry from influencing the feature scale. The fixed chronological, purged, and leave-log-out assignments are detailed in Table 3.

Following the temporal-statistical representation used by AeroTSBoost [2], each normalized channel window was summarized by 18 descriptors: mean, standard deviation, minimum, maximum, range, five quantiles (10th, 25th, 50th, 75th, and 90th), start value, end value, endpoint difference, mean and standard deviation of first differences, mean and maximum absolute first difference, and lag-one autocorrelation. For window w , concatenation over the 87 channels gives

$$\mathbf{f}_w = [d_k(\mathbf{z}_{w,c})]_{k=1,\dots,18;c=1,\dots,87} \in \mathbb{R}^{1566}. \quad (2)$$

The accompanying feature dictionary stores the descriptor, telemetry channel, and PX4 topic for every coordinate of $\mathbf{f}_w$. This explicit provenance enables later attribution to be aggregated from statistical feature to channel, topic, and subsystem without relying on name reconstruction.

3.2. Hierarchical Anomaly Detection and Fault-Domain Diagnosis

HS-AeroTS separates runtime anomaly detection from conditional fault-domain diagnosis. Stage 1 is a class-balanced binary LightGBM model that produces an anomaly score for each feature window. Its operating threshold is selected on the validation precision-recall curve by maximizing F1 and is then frozen for test evaluation. AUPRC and AUROC assess threshold-independent detection, while threshold-dependent F1 and event-level measures assess the operating point.

Stage 2 is a four-class LightGBM model trained only on windows with an explicit anomalous fault-domain label: External Position, Global Position, Altitude, or Mechanical/Electrical. Normal windows and Uncategorized windows are excluded from this conditional training task. Stage 2 uses the same log membership as Stage 1 and is evaluated with Macro-F1, Balanced Accuracy, Macro Recall, and per-class metrics. Core model parameters are reported in Table 3, while frozen machine-readable configurations and run records are part of the analysis package described in the Data Availability Statement.

For the operational Cascade, a window below the Stage 1 threshold is assigned Normal; otherwise, the window is passed to Stage 2. This preserves the distinction between conditional four-class diagnosis and complete five-class runtime prediction. Direct Five-Class is the comparator that predicts Normal plus the four fault domains in one multiclass model. Random Forest, majority, and stratified-random methods are used as reference methods under the corresponding task definitions.

3.3. SHAP-Based Explainable Evidence

For each Stage 2 prediction, TreeSHAP contributions for the predicted class raw score were retained and converted to non-negative evidence magnitudes. For feature j and predicted class $\hat{c}(x)$,

$$e_j(x) = \left| \phi_j^{(\hat{c}(x))}(x) \right|. \quad (3)$$

The feature dictionary defines the aggregation sets. Channel-, topic-, and subsystem-level evidence is obtained by summing feature evidence through the hierarchy:

$$E_q(x) = \sum_{j \in \mathcal{F}(q)} e_j(x), \quad E_t(x) = \sum_{q \in \mathcal{Q}(t)} E_q(x), \quad E_s(x) = \sum_{q \in \mathcal{Q}(s)} E_q(x). \quad (387)$$

Aggregation conservation is checked at each level using

$$\epsilon_L(x) = \sum_{u \in L} E_u(x) - \sum_j e_j(x). \quad (389)$$

Evidence quality is evaluated by validation-derived SHAP top- k masking against equal-size random masking, and by channel-level agreement with a document-derived semantic channel-to-subsystem mapping. Let T_i^K be the K highest-evidence channels for sample i , $g(q)$ the declared subsystem of channel q , and y_i the fault-domain label. We report

$$C_i(K) = \frac{1}{K} \sum_{q \in T_i^K} \mathbb{I}[g(q) = y_i], \quad \text{Consistency@K} = \frac{1}{N} \sum_{i=1}^N C_i(K), \quad (394)$$

$$\text{Hit@K} = \frac{1}{N} \sum_{i=1}^N \mathbb{I} \left[\sum_{q \in T_i^K} \mathbb{I}[g(q) = y_i] > 0 \right]. \quad (395)$$

For non-negative channel evidence $E_q(x_i)$, weighted consistency is

$$\text{WeightedConsistency} = \frac{1}{N} \sum_{i=1}^N \frac{\sum_q E_q(x_i) \mathbb{I}[g(q) = y_i]}{\sum_q E_q(x_i)}. \quad (397)$$

If $Q = 87$ and n_y channels are assigned to domain y , the exact without-replacement random baselines are n_y/Q for Consistency@K and $1 - \binom{Q-n_y}{K} / \binom{Q}{K}$ for Hit@K, averaged over the evaluated labels. The reference mapping is defined from PX4 message topics and field semantics without access to SHAP values. It corrects generic string-token leakage in the original rule, such as attitude yaw and manual-control axes being interpreted as global-position fields; 11 of 87 channel assignments change. Channels assigned “Shared” count as non-matches for every specific fault domain. Because the reference is author-defined rather than independently expert-adjudicated, all of these quantities are mapping sensitivities, not subsystem ground-truth validation. The complete 87-channel mapping is given in Appendix A.

The primary masking test replaces selected test features with feature-wise medians computed from training windows. A sensitivity analysis instead replaces all selected features in each test window with their joint values from one deterministically sampled training-window donor. The donor preserves dependence within the replaced block, although combining it with unreplaced test features can still be off-manifold. Random feature subsets are nested within each model seed. Paired uncertainty is estimated by resampling complete flight logs and averaging the contrast across the five model seeds. These tests assess numerical organization and prediction sensitivity under specified perturbations; they do not establish physical causality or feature necessity.

3.4. PX4 Software-Module Suspect Ranking

The architecture-aware layer maps topic-level SHAP evidence to commit-specific PX4 software-module inspection candidates. Firmware provenance was read from ULog headers. The corpus contained seven firmware commits, and the most frequent commit, 82aa24adfca29321cfd1209e287eab6c2b16780e, occurred in 562 of 1389 usable logs; its

source tree was selected for static analysis. All architecture-aware aggregate results are recomputed only from matching-commit logs. In the fixed test partition, this restriction retains 14,419 of 34,059 windows (42.3%) from 561 matching-commit logs and 1,393 of 3,602 ground-truth anomalous windows (38.7%). Windows from the other six commits are excluded rather than propagated through a mismatched graph.

The source scanner identifies uORB topic occurrences, assigns publisher or subscriber roles using the nearest recognized uORB API hint within a 500-character local context, and converts source paths into module identifiers. An internal consistency check covers all 244 unambiguous same-statement declarations or calls involving the 18 evaluated topics. Within this explicit scope, agreement with the statement-level role rule and recovery of its explicit edges are both 1.000. This is not an independent gold audit because the reference and parser use the same uORB hint vocabulary at different context granularities; it does not quantify recall for macro-generated, indirect, conditionally compiled, or runtime-created relations.

The initial source-tree scan contains 180 edges and 54 modules. A conservative-exclusion sensitivity removes paths containing examples, templates, tests, POSIX, QURT, Bebop, Linux, Snapdragon, simulator, or replay markers for the log-confirmed PX4FMU_V2 and NuttX environment. The filtered edge list must be a strict subset of the original graph and is checked for evidence conservation. This rule is deliberately path-based: neither the unfiltered nor filtered graph is an actual target-specific compiled build graph. Full exclusion rules and software settings are listed in Appendix B.

For propagation mode r , let $\mathcal{M}_t^{(r)}$ be the modules connected to topic t and E_t its topic-level evidence. Evidence is allocated uniformly across the connected modules:

$$A_{t,m}^{(r)} = \begin{cases} E_t / |\mathcal{M}_t^{(r)}|, & m \in \mathcal{M}_t^{(r)}, \\ 0, & \text{otherwise,} \end{cases} \quad S_m^{(r)} = \sum_t A_{t,m}^{(r)}.$$

Producer-only, consumer-only, bidirectional, topic-only, and random-mapping modes represent different engineering interpretations or controls for the same topic evidence. Uniform allocation can favor or dilute modules as a function of topic degree. A further sensitivity divides each module's aggregate score by its number of mapped topics before renormalization. These ablations are reported rather than treating one propagation rule as uniquely valid. The outputs are commit-level source-tree inspection candidates: they are version-dependent, static, propagation-dependent, and neither causal root-cause localization nor deployment-build validation. Mapping coverage, exclusion sensitivity, evidence conservation, and propagation-mode results are reported in Section 5.2.

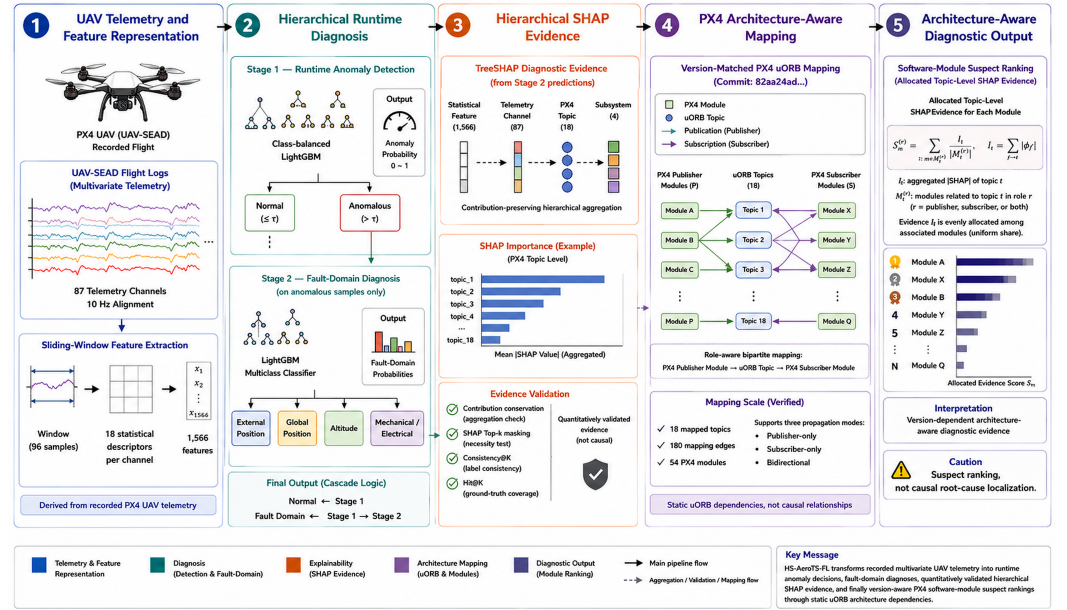


Figure 2. Methodological pipeline. Runtime telemetry is transformed into temporal-statistical windows for anomaly detection and conditional fault-domain diagnosis. TreeSHAP evidence is aggregated through the feature–channel–topic–subsystem hierarchy, while a static source-tree graph reconstructed at the majority PX4 firmware commit maps topic evidence to software-module inspection candidates. Ground-truth-anomaly-conditioned and Stage-1-gated evaluations are reported separately. The architecture branch is associative and version-dependent, not causal or deployment-build validated.

4. Experimental Protocols

4.1. Data Sources and Experimental Roles

UAV-SEAD supplied the supervised training, runtime detection, and fault-domain diagnosis data. ALFA was used only for zero-shot binary anomaly detection, and Uncategorized UAV-SEAD logs were used only for flight-level weak-label screening. Neither analysis expanded the four-class supervised labels. Controlled PX4 replay was kept separate from real-flight model fitting and provided the only module-level mutation ground truth. The roles, protocol sizes, model configurations, and evaluation outputs are summarized in Table 3.

Two recent multivariate time-series methods were also examined as detector references. CATCH [23] used the official model core at repository commit 3647c69be5eb56649b072596cf89098e689e20c3, all 87 raw telemetry channels, and the same fixed chronological train/validation/test windows as HS-AeroTS. Following its unsupervised task definition, each of three seeds trained only on a deterministic sample of 4096 normal training windows; model selection used 1024 normal validation windows. The declared compute budget was two epochs with batch size 8; evaluation batch size 128 changed only inference throughput. This matched-input execution removes the earlier 18-topic aggregation but remains a budget-limited, normal-only comparison rather than an equal-supervision contest. A GCAD-inspired reference [24] used a ridge-regularized linear VAR(1) Granger-prediction score on 8192 normal windows; it is not a reproduction of the published nonlinear GCAD network. These baselines are reported with their actual protocols and are not used to claim unrestricted state-of-the-art superiority.

4.2. Data Splits and Statistical Protocol

The fixed chronological split provides the primary reproduction baseline and preserves the per-log temporal order used by P2 and P3. The purged protocol removes 14 windows

on each side of split boundaries to test sensitivity to temporal proximity. Leave-log-out assigns complete flight logs, rather than individual windows, to train, validation, and test partitions; it is the principal test of generalization to unseen logs. These protocols address different risks and are reported together rather than treated as interchangeable estimates.

The primary real-flight and purged experiments use five frozen model seeds. Thresholds are selected on validation data only. Seed standard deviations quantify variation among the five fitted models; they are not sampling intervals. Absolute Stage 1 AUPRC, Stage 2 Macro-F1, Cascade Macro-F1, and Direct Five-Class Macro-F1 are additionally assigned 95% cluster-bootstrap intervals using 1000 resamples of complete test flight logs with seed 20260825. Each resample is evaluated for all five frozen model seeds and then averaged, so overlapping windows from a flight remain in the same cluster. ALFA intervals resample its 46 sequences and Uncategorized intervals resample its 1017 flights. Paired Cascade-versus-Direct and explanation-masking intervals use the same flight-level principle. Replay intervals use 5000 two-way cluster-bootstrap repetitions that independently resample the three source logs and four mutation types. This preserves the crossed dependence structure, although inference remains low-resolution because each cluster dimension is small. Protocol-specific counts, parameters, software versions, and frozen statistical settings are listed in Table 3 and Appendix B.

OpenAI Codex was used to generate and review inference-only recomputation code, assist with statistical interpretation, and revise manuscript language and structure. The authors executed the code, checked the frozen-input hashes and resulting tables, reviewed all generated text, and take responsibility for the analysis and manuscript. Codex was not used to collect data, train or tune models, add baselines, or create experimental observations.

4.3. Controlled PX4 Replay and Evaluation Layers

The replay protocol fixed the PX4 source commit and Ubuntu 20.04 WSL environment and used system-wide ULog replay without Gazebo, ROS, QGroundControl, or NuttX. Three real ULogs were paired with four source mutations in Commander, EKF2, INAV, and Land Detector. Each log-mutation pair included a matched baseline and fault replay, with injection at 30 s, yielding 24 cases and 12 fault runs.

Because the source ULogs lacked `ekf2_timestamps`, EKF2 used the same generic publication adapter in baseline and fault conditions; only fault activation differed after the injection time. This compatibility detail is retained as a replay limitation. Evaluation separates onset-conditioned module ranking, which uses the known onset and matched baseline, from detector-gated ranking, which first requires the unchanged Stage 1 detector to exceed its validation threshold. Only the latter measures the complete detection-gated evidence chain, and neither is interpreted as causal localization.

Table 3. Experimental protocols, configurations, and evaluation roles. Frozen hyperparameters and per-run records can be supplied for confidential assessment as described in the Data Availability Statement.

Protocol	Role	Configuration and counts	Primary outputs
Fixed chronological	Primary UAV-SEAD reproduction	96-sample windows, stride 8, horizon 12; 1389 logs; 218,537 windows; train/validation/test: 152,340/32,138/34,059; 87 channels, 1566 features; five seeds	Stage 1/2 LightGBM, Cascade, Direct Five-Class; AUPRC, AUROC, Macro-F1, Balanced Accuracy
Purged	Temporal-proximity sensitivity	Same feature/model configuration; 14 windows removed at each boundary; train/validation/test: 133,353/8,536/17,966 windows; five seeds	Same task definitions; protocol sensitivity metrics
Leave-log-out	Unseen-log generalization	Same feature/model configuration; 970/206/213 complete logs for train/validation/test; 149,774/37,324/31,439 windows; 1000 flight-log bootstraps	Detection, diagnosis, and hierarchy comparison
ALFA	Zero-shot external binary reference	16 semantically shared channels; no four-class label transfer; UAV-SEAD validation threshold	Binary AUPRC and AUROC only
Uncategorized	Flight-level weak-label screening	139 Uncategorized logs versus 898 pure-Normal logs; no temporal event labels	Flight-level screening metrics only
CATCH	Budget-constrained feasibility reference	Official model core; all 87 raw channels; fixed chronological split; 4096 normal training windows; two epochs; three seeds	AUPRC, AUROC, validation-threshold F1, and event metrics; no superiority claim
PX4 replay	Source-mutation module ground truth	3 ULogs $\times$ 4 mutations $\times$ baseline/fault; 24 cases, 12 fault runs; 30 s injection; 5000 two-way cluster bootstraps	Onset-conditioned and detector-gated Top- k , MRR, and EXAM

Core model configurations: Stage 1 LightGBM uses 1600 trees, learning rate 0.025, 64 leaves, minimum child size 80, class-balanced weighting, and 80-round early stopping. Stage 2 LightGBM uses 1200 trees with the same learning rate, 64 leaves, class-balanced weighting, and minimum child size 30; Direct Five-Class uses the Stage 2 configuration with five labels. The Random Forest reference uses 500 trees, square-root feature subsampling, minimum leaf size 2, and balanced subsampling.

5. Results

5.1. Real-Telemetry Detection and Fault-Domain Diagnosis

The audited UAV-SEAD corpus contained 1396 mapped logs, of which 1389 were retained as usable after excluding missing, unparsable, or corrupt records. The retained logs yielded 1,892,063 aligned records at 10 Hz and 87 core telemetry channels. Applying the fixed AeroTSBoost-compatible representation produced 218,537 overlapping windows with 1566 temporal-statistical features per window, including 15,694 anomalous windows. Under the fixed per-log chronological split, 152,340, 32,138, and 34,059 windows were assigned to training, validation, and test partitions, respectively; the corresponding anomalous-window

counts were 9011, 3081, and 3602. These audit values matched the expected dataset and representation counts before model evaluation.

The five-seed LightGBM detector reproduced the reference performance level on the fixed chronological test partition. The mean AUROC was 0.9512 ± 0.0006 and the mean AUPRC was 0.7522 ± 0.0039 , with a 95% flight-log cluster-bootstrap CI of [0.6915, 0.8080]. The independently optimized test-set F1 was 0.6924 ± 0.0056 ; this is an oracle descriptive value and is not an operational estimate because its threshold uses test labels. These values were close to the reference values of 0.9516, 0.7516, and 0.6860, respectively, and preserved the reported detector performance trend. The AUPRC result is the primary threshold-independent measure because anomalous windows were a minority of the evaluated windows.

For operational use, the threshold was selected separately for each seed by maximizing validation F1 and then frozen before test evaluation. With this validation-derived threshold, the test F1 was 0.6895 ± 0.0067 , and the log-aware event F1 was 0.6051 ± 0.0218 . Thus, the threshold-dependent operating result remained close to the independently optimized test F1, while the event-level measure was lower because it evaluates the temporal detection behavior at the frozen operating point.

The matched-input CATCH execution used the official model core, all 87 raw channels, and the fixed chronological split. Across three seeds, test AUPRC was 0.2094 ± 0.0038 , AUROC was 0.7318 ± 0.0052 , validation-threshold F1 was 0.2992 ± 0.0038 , and log-aware event F1 was 0.2537 ± 0.0040 . The GCAD-inspired linear VAR(1) ablation produced AUPRC of 0.1991. These are budget-constrained feasibility references, not competitive benchmark estimates: CATCH followed a normal-only unsupervised objective with 4096 sampled training windows and two epochs, whereas the GCAD result is not the published nonlinear architecture. Their numerical values therefore document the implemented resource-constrained protocols and do not support detector-superiority or state-of-the-art claims.

On the 3602 anomalous test windows, the anomaly-only Stage 2 LightGBM classifier attained Macro-F1 of 0.8980 ± 0.0020 (95% flight-log CI [0.8581, 0.9296]) and Balanced Accuracy of 0.8863 ± 0.0021 . Its mean per-class recall and F1 were 0.9685 and 0.9390 for External Position, 0.9279 and 0.9162 for Global Position, 0.7916 and 0.8503 for Altitude, and 0.8574 and 0.8865 for Mechanical/Electrical, respectively. The corresponding Random Forest baseline achieved Macro-F1 of 0.8724 ± 0.0037 and Balanced Accuracy of 0.8382 ± 0.0052 . The majority and stratified-random references achieved Macro-F1 values of 0.1608 and 0.2358 ± 0.0090 . These comparisons indicate that the four-class diagnostic task was learned under the specified ground-truth-anomaly-conditioned label protocol; they do not represent complete online five-class performance.

When Stage 1 thresholding and Stage 2 prediction were combined, the complete five-class Cascade achieved Macro-F1 of 0.6902 ± 0.0071 (95% flight-log CI [0.6308, 0.7350]) and Balanced Accuracy of 0.6755 ± 0.0152 . Its mean recall/F1 pairs were 0.9629/0.9632 for Normal, 0.7787/0.7088 for External Position, 0.6649/0.6493 for Global Position, 0.4329/0.5186 for Altitude, and 0.5379/0.6112 for Mechanical/Electrical. The lower anomaly-class performance relative to the conditional Stage 2 result reflects the additional Stage 1 gating step. The five-seed Direct Five-Class LightGBM comparator achieved Macro-F1 of 0.6599 ± 0.0029 (95% flight-log CI [0.5991, 0.7070]) and Balanced Accuracy of 0.6144 ± 0.0021 on the same fixed chronological test partition. These fixed-split values describe the primary reproduction protocol; stricter isolation results are reported separately.

5.2. SHAP Evidence and Software-Module Suspect Ranking

The primary explanation analysis was ground-truth-anomaly-conditioned: it used all 3602 labelled anomalous test windows from the fixed chronological protocol and did

not require Stage 1 to detect them. For each window, the TreeSHAP contributions of the predicted Stage 2 class were converted to absolute evidence and aggregated from statistical features to telemetry channels, PX4 topics, and subsystems. The aggregation preserved the total evidence mass: the maximum absolute conservation error across the hierarchy was 7.11×10^{-15} . This verifies only the numerical redistribution of evidence; it does not establish that the attributed channel or subsystem is the physical source of the anomaly.

The masking analysis tested whether the validation-derived SHAP ranking was more informative than an equal-size random feature selection. The top- k SHAP features were ranked on validation anomaly windows and replaced in the test data by feature-wise medians calculated from the training windows. The Macro-F1 decrease for SHAP masking increased from 0.0409 at $k = 10$ to 0.2683 at $k = 100$. Across 20 random subsets nested within each of five model seeds, the paired SHAP-minus-random advantage was 0.0404 (95% flight-log-bootstrap CI [0.0102, 0.0713]) at $k = 10$ and 0.2646 [0.2052, 0.3300] at $k = 100$. Replacing the selected feature block with values from sampled training rows produced drops of 0.0516 [0.0277, 0.0752] and 0.4955 [0.4604, 0.5283], respectively. The direction therefore persisted under both median and joint-donor perturbations, although donor replacement can still create cross-block combinations with unreplaced features. These results support prediction-relevant feature concentration, not causal influence or physical necessity.

At the telemetry-channel level, the original string-rule mapping and the document-derived mapping agreed on 76 of 87 channels. Under the document-derived reference, overall Consistency@1 was 0.2500 ± 0.0056 , compared with the exact random baseline of 0.1707. Across $K \in \{1, 3, 5, 10\}$, Consistency@K was 0.2500 ± 0.0056 , 0.3494 ± 0.0172 , 0.3410 ± 0.0051 , and 0.2775 ± 0.0028 , respectively. Class-level Consistency@1 was 0.0363 for External Position, 0.0349 for Global Position, 0.3779 for Altitude, and 0.8788 for Mechanical/Electrical; their respective random baselines were 0.0345, 0.1379, 0.1724, and 0.5402. Thus, External Position remained near random and Global Position fell below random after correcting semantically inappropriate axis-token assignments. The reduction from the original overall value of 0.3705 demonstrates material mapping sensitivity. Consistency@K is therefore not used as evidence of explanation correctness; it only describes agreement with a declared semantic reference.

For the architecture-aware layer, the unfiltered static scan at commit 82aa24adfca29321cfd1209e287eab6c2b16780e contained 180 topic-role-module edges covering 54 PX4 modules. The 244-occurrence internal consistency check returned 1.000 agreement and edge recovery within its shared-vocabulary scope; it is not an independent uORB gold audit. Conservative source-path exclusion removed 35 edges and 18 modules, leaving 145 edges, 36 modules, and all 18 topics while preserving evidence conservation. Producer Top-3 overlap was 3/3 before versus after filtering, while Top-5 overlap was 4/5; bidirectional Top-5 was unchanged. Under filtered producer-only propagation, the highest-ranked source-tree inspection candidates were `src/modules/mavlink` (SHAP share 0.2572), `src/modules/ekf2` (0.2318), `src/modules/sensors` (0.1243), `src/modules/vtol_att_control` (0.0811), and `src/modules/position_estimator_inav` (0.0708). Figure 3(d) shows this conservative-exclusion result. The exclusion is path-based and does not reconstruct the compiled PX4FMU_V2 build; degree normalization and propagation-mode sensitivity further preclude interpreting the order as a stable root-cause ranking.

The separate Stage-1-gated matching-commit analysis used the frozen validation-derived detector thresholds. Averaged across the five seeds, the 14,419 matching-commit test windows comprised 937.2 true positives, 455.8 false negatives, 532.2 false positives, and 12,493.8 true negatives; mean detector recall was 0.6728 ± 0.0248 and precision was 0.6398 ± 0.0305 . Thus, only the true-positive subset can carry detected real-anomaly evidence

downstream, while the false-negative subset is absent from a deployed gate. The false-positive windows were assigned by Stage 2 to External Position (128.8), Global Position (216.4), Altitude (88.4), and Mechanical/Electrical (98.6) on average. These are predicted suspect categories for false alarms, not validated fault labels. This gated analysis is reported separately from the 1393-window ground-truth-anomaly-conditioned analysis.

The controlled-replay propagation ablation is reported in Table 4. Bidirectional propagation achieved Top-1/3/5 of 0.250/0.333/0.333 and MRR of 0.323, while producer-only and topic-only achieved higher Top-3 and Top-5 values of 0.500. The random-mapping control was lower, but the confidence intervals were wide and overlapping. With only 12 dependent mutation runs, the ablation does not identify a uniquely superior propagation rule.

Table 4. Controlled-replay propagation ablation with 95% two-way cluster-bootstrap confidence intervals. The bootstrap independently resamples the three source logs and four mutation types; the intervals remain low-resolution because both cluster counts are small.

Mode	Top-1 (95% CI)	Top-3 (95% CI)	Top-5 (95% CI)	MRR (95% CI)
Bidirectional	0.250 [0.000, 0.583]	0.333 [0.000, 0.750]	0.333 [0.000, 0.750]	0.323 [0.046, 0.679]
Producer-only	0.250 [0.000, 0.583]	0.500 [0.167, 0.833]	0.500 [0.167, 0.833]	0.414 [0.143, 0.690]
Consumer-only	0.083 [0.000, 0.333]	0.167 [0.000, 0.500]	0.250 [0.000, 0.583]	0.182 [0.035, 0.510]
Topic-only	0.167 [0.000, 0.500]	0.500 [0.167, 0.833]	0.500 [0.167, 0.833]	0.373 [0.147, 0.625]
Random mapping	0.083 [0.000, 0.333]	0.083 [0.000, 0.333]	0.167 [0.000, 0.500]	0.150 [0.031, 0.410]

Figure 3 summarizes the masking comparison, document-derived semantic-map agreement, class-specific Consistency@1, and the conservatively filtered commit-level source-tree candidates. Selected frozen analysis records are described in the Data Availability Statement.

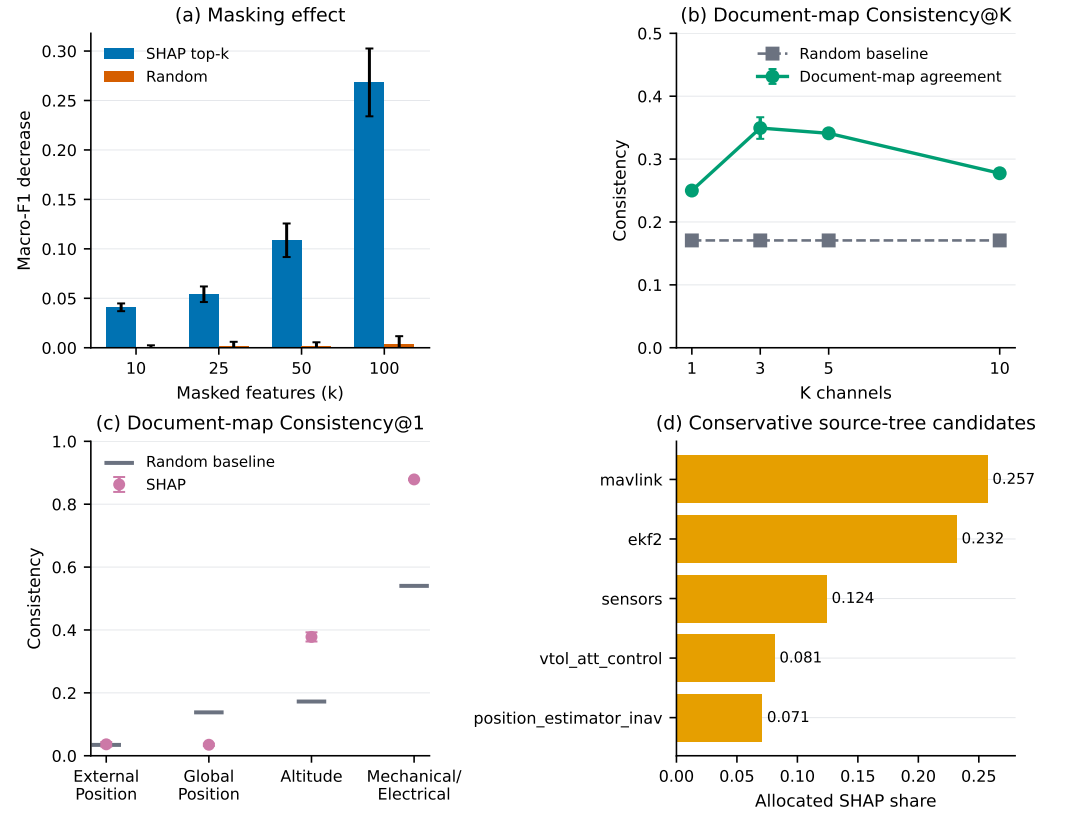


Figure 3. SHAP evidence and architecture-aware inspection candidates under the fixed chronological protocol. (a) Macro-F1 decrease after masking validation-ranked SHAP features or equal-size random subsets; bars show means and error bars show seed standard deviations. (b) Overall agreement with the document-derived semantic mapping and exact without-replacement random baselines; error bars show seed standard deviations. (c) Class-specific Consistency@1; error bars show seed standard deviations. (d) Filtered producer-only source-tree candidates computed from the 1393 ground-truth anomalous test windows whose firmware matches the selected PX4 commit; bars are mean normalized shares across seeds and no interval is shown. Panel (d) is not a target-build or causal localization result.

5.3. Strict Validation, External Validation, and Capability Boundaries

The stricter protocols reduced fixed chronological performance and changed the numerical ordering of the complete five-class models. Across five seeds under the 14-window purged protocol, Stage 1 AUPRC was 0.6006 ± 0.0062 , Stage 2 Macro-F1 was 0.7944 ± 0.0049 , Cascade Macro-F1 was 0.5683 ± 0.0177 , and Direct Five-Class Macro-F1 was 0.6024 ± 0.0049 . The paired Direct-minus-Cascade difference was 0.0340, with a flight-log-bootstrap 95% CI of $[-0.0047, 0.0762]$. The mean ordering reversed relative to the fixed chronological split, but the paired interval included zero and therefore did not support a stable purged-protocol advantage. Figure 4 summarizes these protocol-level comparisons.

Leave-log-out evaluation assigned complete logs to the partitions, with 970 logs for training, 206 for validation, and 213 for testing. Across five seeds, Stage 1 AUPRC was 0.6296 ± 0.0023 and AUROC was 0.9265 ± 0.0008 . At the validation-derived operating threshold, Stage 1 F1 was 0.5989 ± 0.0042 and log-aware event F1 was 0.4533 ± 0.0241 . The anomaly-only Stage 2 LightGBM classifier achieved Macro-F1 of 0.9041 ± 0.0029 and Balanced Accuracy of 0.8876 ± 0.0040 . On the complete five-class task, the Cascade achieved Macro-F1 of 0.5962 ± 0.0048 and Balanced Accuracy of 0.5840 ± 0.0057 , while Direct Five-Class achieved Macro-F1 of 0.6092 ± 0.0043 and Balanced Accuracy of 0.6341 ± 0.0022 .

The paired Direct-minus-Cascade Macro-F1 difference under leave-log-out was 0.0130, with a flight-log bootstrap 95% confidence interval of $[-0.0282, 0.0438]$ based on 1000

repetitions. Because the interval includes zero, these results do not support a stable performance difference or a general performance advantage for the hierarchical Cascade. The Cascade remains a distinct gated decision structure, but its fixed-split performance ordering should not be interpreted as a protocol-independent superiority.

The ALFA experiment evaluated zero-shot binary anomaly detection using 16 semantically shared physical channels and a threshold fixed from UAV-SEAD validation data. On 46 ALFA sequences and 5273 ALFA windows, the transferred detector obtained AUPRC of 0.3181 ± 0.0086 (95% sequence-cluster CI [0.2540, 0.3956]) and AUROC of 0.6679 ± 0.0069 . The ALFA faults were not mapped to the four UAV-SEAD fault domains; these values therefore assess binary score transfer only. The separate flight-level screening of 139 Uncategorized logs against 898 pure-Normal logs produced flight-level AUPRC of 0.2784 ± 0.0046 (95% flight-cluster CI [0.2086, 0.3664]), AUROC of 0.6610 ± 0.0049 , F1 of 0.3307 ± 0.0142 , Uncategorized recall of 0.4118 ± 0.0314 , and Normal specificity of 0.8575 ± 0.0073 . Because Uncategorized annotations have no temporal ranges, this screening does not support point-wise or event-level claims.

Table 5 separates model-seed standard deviations from sampling uncertainty for the principal absolute metrics. The cluster intervals are wider than the seed dispersions because logs or sequences, rather than highly overlapping windows, are the resampling units.

Table 5. Principal absolute metrics with model-seed standard deviation and 95% cluster-bootstrap sampling intervals (1000 repetitions; seed 20260825). UAV-SEAD intervals resample complete test flight logs, ALFA resamples sequences, and Uncategorized resamples flights.

Dataset/protocol	Metric	Point estimate	Seed SD	95% cluster CI
UAV-SEAD fixed	Stage 1 AUPRC	0.7522	0.0039	[0.6915, 0.8080]
UAV-SEAD fixed	Stage 2 Macro-F1	0.8980	0.0020	[0.8581, 0.9296]
UAV-SEAD fixed	Cascade Macro-F1	0.6902	0.0071	[0.6308, 0.7350]
UAV-SEAD fixed	Direct Five-Class Macro-F1	0.6599	0.0029	[0.5991, 0.7070]
UAV-SEAD purged	Stage 1 AUPRC	0.6006	0.0062	[0.4968, 0.7109]
UAV-SEAD purged	Stage 2 Macro-F1	0.7944	0.0049	[0.7040, 0.8489]
UAV-SEAD purged	Cascade Macro-F1	0.5683	0.0177	[0.4532, 0.6338]
UAV-SEAD purged	Direct Five-Class Macro-F1	0.6024	0.0049	[0.4890, 0.6664]
UAV-SEAD leave-log-out	Stage 1 AUPRC	0.6296	0.0023	[0.4945, 0.7456]
UAV-SEAD leave-log-out	Stage 2 Macro-F1	0.9041	0.0029	[0.8276, 0.9549]
UAV-SEAD leave-log-out	Cascade Macro-F1	0.5962	0.0048	[0.4889, 0.6799]
UAV-SEAD leave-log-out	Direct Five-Class Macro-F1	0.6092	0.0043	[0.5091, 0.6836]
ALFA zero-shot	Stage 1 AUPRC	0.3181	0.0086	[0.2540, 0.3956]
Uncategorized screening	Flight-level AUPRC	0.2784	0.0046	[0.2086, 0.3664]

The controlled PX4 replay provided the only module-level ground truth, but its evaluation exposed a detector-gated capability boundary. In the onset-conditioned bidirectional analysis, which supplied the known 30-s injection time and used matched baseline evidence, overall Top-1, Top-3, and Top-5 recalls were 0.250 (95% two-way cluster-bootstrap CI [0.000, 0.583]), 0.333 [0.000, 0.750], and 0.333 [0.000, 0.750], respectively. MRR was 0.3227 [0.0455, 0.6790], and mean EXAM was 0.3318 [0.1065, 0.5772]. These intervals independently resample the three source logs and four mutation types and remain wide because both cluster counts are small. The conditional ranks were uneven across mutations: EKF2 innovation bias reached Top-1 in two of three runs, whereas INAV local-z freeze and Land Detector state inversion did not enter Top-5 in any run. These results quantify conditional ranking behavior after onset information is supplied.

The unchanged real-flight Stage 1 detector, however, detected 0 of 12 controlled mutation runs at its fixed threshold of 0.3374776883. Consequently, detector-gated Top-1, Top-3, Top-5, and MRR were all zero; detection delay and detector-gated ranking delay were undefined. Figure 5 contrasts the onset-conditioned and detector-gated metrics. The onset-conditioned values must therefore not be presented as end-to-end software-module localization performance. Taken together, the replay results support only conditional

source-tree inspection ranking while identifying real-flight-to-replay domain shift as a current limit of the complete evidence chain.

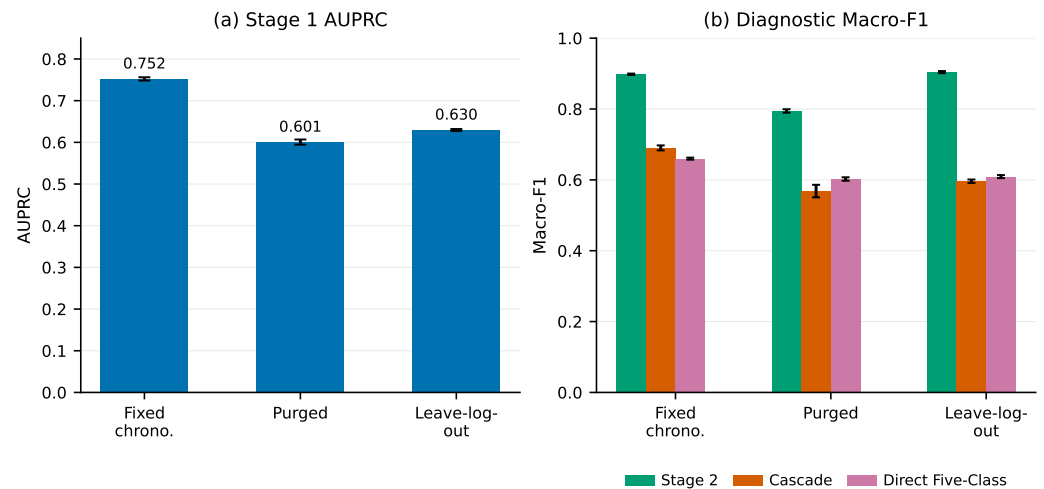


Figure 4. Primary protocol comparison for Stage 1 detection and complete or conditional diagnosis. The fixed chronological, purged, and leave-log-out protocols are evaluated with their respective split definitions. In every panel, error bars show standard deviations across five frozen model seeds, not sampling confidence intervals; absolute flight-log cluster intervals are reported in Table 5. Purging reverses the mean ordering of Cascade and Direct Five-Class Macro-F1, but the paired flight-log interval for their difference includes zero.

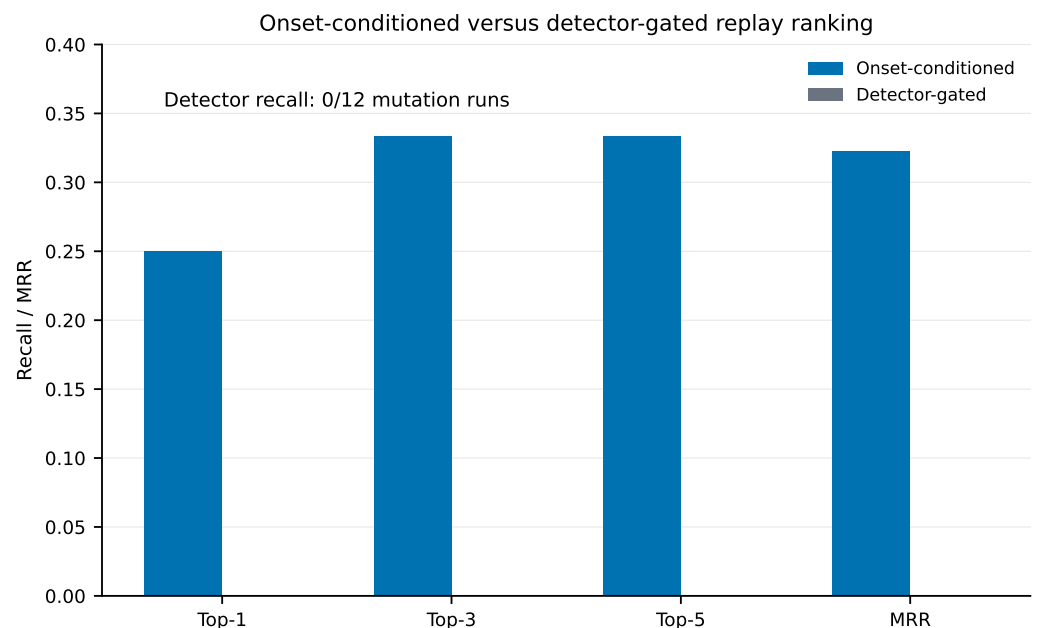


Figure 5. Controlled PX4 replay module-ranking metrics under two evaluation layers. Onset-conditioned ranking uses the known injection time and matched baseline evidence; detector-gated ranking first requires the unchanged real-flight Stage 1 detector to fire. The detector-gated recall was 0/12 for the evaluated mutation runs.

6. Discussion

6.1. Principal Findings

The results answer the three research questions at different levels of evidence. For RQ1, real PX4 telemetry supported runtime anomaly detection and, conditional on an

anomalous window having a specific label, four-class fault-domain diagnosis. The fixed chronological protocol yielded strong detector and Stage 2 results, whereas the lower leave-log-out detector AUPRC showed that this capability is sensitive to flight-log isolation. The high anomaly-only Stage 2 Macro-F1 should therefore be interpreted as evidence that the labeled fault domains are distinguishable after anomaly selection, not as the performance of a complete online diagnostic system.

For RQ2, the hierarchical SHAP analysis provided quantitatively testable diagnostic evidence rather than only qualitative feature-importance examples. Evidence mass was conserved across feature, channel, topic, and subsystem aggregation, and masking the validation-ranked SHAP features degraded diagnostic performance substantially more than masking equal-size random subsets. However, the document-derived mapping sensitivity materially reduced overall Consistency@1, left External Position near random, and placed Global Position below its class-specific random baseline. The explanation layer is consequently useful for organizing model-relevant telemetry evidence, but semantic agreement is mapping-dependent and cannot be treated as uniform explanation validation.

The majority-commit uORB graph extended this evidence into commit-level source-tree inspection candidates and thereby partially addressed the architecture-aware part of RQ2. The ground-truth-anomaly-conditioned aggregation was restricted to 1393 anomalous test windows whose firmware matched the graph. Under frozen Stage 1 gating, an average of 937.2 of those windows were detected, 455.8 were missed, and 532.2 false-positive matching-commit windows entered the downstream branch. The 1.000 parser result is only an internal consistency check, not an independent gold audit. Conservative path exclusion reduced the graph from 180 to 145 edges and from 54 to 36 modules while retaining all topics, but did not reconstruct the actual compiled target. Degree normalization and publisher/subscriber semantics further show that the candidates prioritize source-tree inspection rather than identify a defective module.

For RQ3, strict isolation, external data, and controlled replay exposed complementary limits. Purged and leave-log-out results reduced the apparent robustness of the fixed chronological evaluation; ALFA supported only partial zero-shot transfer of the binary detector; and the controlled mutations were not detected by the unchanged real-flight detector. Taken together, HS-AeroTS provides traceable evidence at separately evaluated stages, but the evaluated chain does not establish reliable end-to-end software fault localization or deployment readiness.

6.2. Interpretation of the Hierarchical Model under Strict Protocols

The hierarchy separates two engineering questions that have different label spaces: whether a telemetry window is abnormal and which labeled fault domain best describes an abnormal window. This separation makes the decision path explicit and permits fault-domain-specific evidence to be generated only after an anomaly decision. It also clarifies why Stage 2 performance and complete Cascade performance differ. Stage 2 is evaluated on anomaly-only windows, whereas the Cascade inherits false negatives and false positives from the detector before applying the fault-domain classifier.

The protocol comparisons show that this interpretability advantage should not be conflated with predictive superiority. On the fixed chronological split, the paired Direct-minus-Cascade Macro-F1 contrast was -0.0303 (95% flight-log-bootstrap CI $[-0.0524, -0.0086]$), favoring the Cascade for that split. Purging reversed the mean ordering, but its paired interval included zero; the leave-log-out interval likewise included zero. The available evidence thus does not support a protocol-independent advantage for either complete five-class formulation. The contribution of the hierarchy lies instead in its explicit gating

semantics, its separation of detector and diagnostic errors, and its traceable explanation structure.

The degradation from the fixed chronological protocol to stricter isolation also has methodological implications. Overlapping windows from the same flight can share local operating conditions even when assigned chronologically, and a boundary purge removes only part of this proximity. Assigning entire flight logs to disjoint partitions provides a stronger estimate of transfer to unseen flights. The lower leave-log-out Stage 1 AUPRC and event F1 indicate that the detector is the principal bottleneck in the complete chain, whereas the stable anomaly-only Stage 2 result suggests that labeled fault-domain structure remains learnable once abnormal windows are supplied. Future model comparisons should therefore retain flight-log-level isolation and report detector, conditional diagnostic, and complete-task metrics separately.

The full-channel CATCH execution closes the earlier input-mismatch concern but not every comparability limitation. It used the official model core, the same 87 raw channels and chronological windows, three seeds, and both window- and event-level metrics; however, its normal-only objective, 4096-window training sample, and two-epoch budget differ intrinsically from the supervised LightGBM protocol. The GCAD-inspired result additionally remains a non-official linear ablation. They are therefore feasibility references under declared resource budgets and cannot support a comparative rank, detector-superiority claim, or state-of-the-art claim.

6.3. *Controlled-Replay Negative Result and Capability Boundary*

The replay experiment distinguishes conditional ranking ability from end-to-end diagnostic ability. When the known injection onset and a matched baseline were supplied, the topic-to-module evidence placed the injected module within the Top-1 for some EKF2 runs, but performance was uneven across mutation types and the aggregate Top-5 recall remained 0.333. This onset-conditioned analysis tests whether a localized change in telemetry evidence can be propagated toward a relevant module after temporal alignment has already been provided. It does not test whether the deployed chain can independently recognize the change.

The detector-gated result provides that stronger test and constitutes the main capability boundary of the current system. The unchanged real-flight detector triggered on none of the 12 mutation runs, so all detector-gated ranking metrics were zero. This failure cannot be offset by the onset-conditioned ranks because an online chain cannot produce a timely module suspect when its first stage does not identify the input as anomalous. Reporting the two evaluation layers separately prevents privileged onset information from being mistaken for operational localization performance.

A plausible interpretation is distribution mismatch between the real-flight annotations used to train Stage 1 and the controlled source mutations used in replay. The two settings differ in anomaly mechanisms, signal manifestations, and replay conditions, and the replay benchmark was not used to adapt the detector. This interpretation remains a hypothesis rather than a quantified decomposition of domain shift. The negative result nevertheless establishes that evidence propagation alone is insufficient: end-to-end module diagnosis requires a detector whose operating distribution includes, or generalizes to, the relevant software-induced behaviors.

6.4. *Limitations and Future Work*

The study has several limitations. First, UAV-SEAD provides runtime anomaly and fault-domain annotations rather than comprehensive source-module ground truth; consequently, the real-flight experiments cannot validate causal module localization. Second,

the document-derived channel-to-subsystem reference is independent of SHAP values but remains author-defined rather than independently expert-adjudicated. Its sensitivity results show that semantic-map agreement is not robust for External or Global Position. Third, no independent uORB gold audit is available: the reported 1.000 value is an internal check sharing the parser's hint vocabulary. The conservative exclusion graph remains a source-path approximation rather than an actual PX4 target-build graph and cannot measure macro-generated, indirect, conditionally compiled, or runtime-created relations. Uniform propagation is also sensitive to topic degree. The version-coherent analysis applies only to the 38.7% matching-commit anomaly subset and omits runtime activation, scheduling, message rates, parameter-dependent paths, and indirect physical feedback. Fourth, the controlled benchmark contains only three source logs, four mutation types, and 12 fault runs, and the EKF2 case required a generic replay publisher because the source logs lacked `ekf2_timestamps`. Two-way cluster-bootstrap intervals remain wide because both cluster dimensions are small. Fifth, the available external datasets support limited validation roles: ALFA does not establish four-domain transfer, and Uncategorized logs have no temporal ranges for point-wise or event-level evaluation. Sixth, recent deep baselines were not fairly retrained at comparable budgets and objectives, so CATCH and the GCAD-inspired ablation remain feasibility references. Finally, the analysis source code and machine-readable Supplement are not public; this reduces independent reproducibility even though the article reports the semantic map, software versions, filters, counts, and statistical settings and selected frozen artifacts can be supplied confidentially.

The explanation analysis also requires class-specific caution. Conservation verifies numerical bookkeeping, while masking supports prediction relevance only under the specified perturbations. External Position agreement remained close to random and Global Position fell below random under the document-derived map. These weaknesses may reflect correlated signals, indirect effects across subsystems, broad diagnostic labels, or attribution limitations, but the present experiments do not isolate them. Consistency@K should therefore remain a sensitivity result rather than a validity criterion.

Future work should expand the present matching-commit analysis into per-commit uORB graphs, obtain independent expert adjudication of the semantic subsystem mapping, extend the parser audit to macro-generated and runtime-created relations, and enlarge the mutation benchmark across independent source logs, PX4 versions, modules, and fault mechanisms, with separate development and test mutations. Detector adaptation could then be studied using only development-domain replay data while preserving untouched real-flight and mutation test sets. Dynamic evidence—including runtime uORB activity, execution traces, and temporal propagation—could complement the static graph, provided that its added information is evaluated against known injected modules. These steps would test whether the current architecture-aware suspect ranking can progress from a useful inspection aid toward reliable detector-gated module diagnosis without overstating causal certainty.

7. Conclusions

7.1. Summary and Outlook

This study addressed the gap between detecting abnormal UAV telemetry and producing diagnostic evidence that can guide engineering inspection. HS-AeroTS organizes this process as a hierarchy: a first-stage detector identifies runtime anomalies, a second-stage classifier assigns detected windows to four fault domains, TreeSHAP contributions are aggregated into channel, topic, and subsystem evidence, and a static PX4 uORB source-tree graph propagates matching-commit topic evidence into software-module inspection candidates. Ground-truth-anomaly-conditioned explanations, Stage-1-gated real-flight

analysis, and detector-gated replay are reported separately to preserve the distinction between conditional evidence and complete-system behavior.

The experiments support the utility of the hierarchy within this defined scope. On the fixed chronological UAV-SEAD protocol, Stage 1 achieved an AUPRC of 0.7522 ± 0.0039 (95% flight-log CI [0.6915, 0.8080]), and the ground-truth-anomaly-conditioned Stage 2 classifier achieved a Macro-F1 of 0.8980 ± 0.0020 [0.8581, 0.9296]. Among matching-commit test windows, frozen Stage 1 gating retained an average of 937.2 true anomalous windows, missed 455.8, and admitted 532.2 false positives. SHAP aggregation conserved evidence mass and SHAP-ranked masking caused greater degradation than random masking, but semantic-map agreement was weak for External and Global Position. Conservative path exclusion left 145 edges, 36 modules, and all 18 topics; this remained a source-tree sensitivity rather than a compiled target-build graph.

The strict evaluations delimit these positive findings. Purged and leave-log-out isolation reduced complete-task performance relative to the fixed chronological protocol, and the leave-log-out comparison did not support a stable Macro-F1 difference between the Cascade and Direct Five-Class formulations. More importantly, the unchanged real-flight detector identified none of the 12 controlled mutation runs. Consequently, the nonzero onset-conditioned module ranks cannot be interpreted as detector-gated end-to-end localization performance. Static topic-to-module propagation provides version-dependent inspection evidence, but it neither proves fault causality nor validates an end-to-end module diagnosis capability.

Future work should prioritize a larger mutation benchmark spanning independent source logs, PX4 versions, software modules, and fault mechanisms, with development and test mutations kept separate. Such a benchmark would permit detector adaptation to be evaluated without contaminating real-flight or mutation test sets. Combining the current static uORB evidence with runtime topic activity, execution traces, and temporal propagation may further improve suspect ranking, but these extensions should continue to be assessed against known injected modules and under detector-gated protocols. These steps are necessary to determine whether the present architecture-aware evidence chain can advance from a traceable inspection aid toward dependable module-level diagnosis.

Author Contributions: Conceptualization, Jiao Wu, Chenglie Du and Junsheng Wu; methodology, Jiao Wu and Bo Liu; software, Jiao Wu and Bo Liu; validation, Jiao Wu, Bo Liu and Yilan Li; formal analysis, Jiao Wu, Bo Liu and Yilan Li; investigation, Jiao Wu; data curation, Jiao Wu; writing—original draft preparation, Jiao Wu; writing—review and editing, all authors; visualization, Jiao Wu; supervision, Chenglie Du and Junsheng Wu; project administration, Chenglie Du and Junsheng Wu. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The UAV-SEAD data used for the primary experiments and weak-label screening are publicly available from Hugging Face at <https://huggingface.co/datasets/aykutkabaoglu/uav-flight-anomaly-dataset> under the Creative Commons Attribution 4.0 International license; the analyses used repository revision 774a336a1d1d06b248927a35c2ea1f6869e9e1a4. The processed ALFA data used for zero-shot external validation are publicly available from Figshare at <https://figshare.com/articles/dataset/12707963> (article 12707963, file 24095870) under the Creative Commons Attribution 4.0 International license. The controlled replay experiments used PX4-Autopilot source code at commit 82aa24adfca29321cfd1209e287eab6c2b16780e, available at <https://github.com/PX4/PX4-Autopilot/commit/82aa24adfca29321cfd1209e287eab6c2b16780e>. No new primary flight data were collected. The authors' analysis source code and a public machine-

readable Supplement are not provided. Aggregate numerical results, the complete semantic mapping, software versions, exclusion rules, split counts, and statistical settings are included in the article and its appendices. Subject to third-party licenses, selected frozen configurations, split manifests, the 180-edge table, per-seed predictions, and derived machine-readable reports may be obtained from the corresponding author on reasonable request for confidential editorial or peer-review assessment.

Acknowledgments: During the preparation of this manuscript and study, the authors used OpenAI Codex (GPT-5-based Codex agent, accessed in August 2026) to generate inference-only recalculation code, assist with statistical interpretation, and revise the language and structure of the manuscript. The authors executed and checked the analyses, reviewed and edited the output, and take full responsibility for the content of this publication. No individuals or organizations are otherwise acknowledged.

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A. Document-Derived Semantic Channel Mapping

Table A1 gives the complete mapping used for the semantic agreement sensitivity. It was defined from PX4 topic and field semantics without access to SHAP values. “Shared” is a contextual or cross-domain assignment and is counted as a non-match for every specific fault-domain label. Each channel contributes 18 temporal–statistical features.

Table A1. Complete 87-channel document-derived semantic mapping.

Telemetry channel	PX4 topic	Declared subsystem
actuator_controls_0.control[0]	actuator_controls_0	Mechanical/Electrical
actuator_controls_0.control[1]	actuator_controls_0	Mechanical/Electrical
actuator_controls_0.control[2]	actuator_controls_0	Mechanical/Electrical
actuator_controls_0.control[3]	actuator_controls_0	Mechanical/Electrical
actuator_outputs.output[0]	actuator_outputs	Mechanical/Electrical
actuator_outputs.output[1]	actuator_outputs	Mechanical/Electrical
actuator_outputs.output[2]	actuator_outputs	Mechanical/Electrical
actuator_outputs.output[3]	actuator_outputs	Mechanical/Electrical
actuator_outputs.output[4]	actuator_outputs	Mechanical/Electrical
actuator_outputs.output[5]	actuator_outputs	Mechanical/Electrical
actuator_outputs.output[6]	actuator_outputs	Mechanical/Electrical
actuator_outputs.output[7]	actuator_outputs	Mechanical/Electrical
battery_status.current_a	battery_status	Mechanical/Electrical
battery_status.current_filtered_a	battery_status	Mechanical/Electrical
battery_status.discharged_mah	battery_status	Mechanical/Electrical
battery_status.remaining	battery_status	Mechanical/Electrical
battery_status.voltage_filtered_v	battery_status	Mechanical/Electrical
battery_status.voltage_v	battery_status	Mechanical/Electrical
cpuload.load	cpuload	Mechanical/Electrical
cpuload.ram_usage	cpuload	Mechanical/Electrical
distance_sensor.current_distance	distance_sensor	Altitude
ekf2_innovations.hagl_innov	ekf2_innovations	Altitude
ekf2_innovations.heading_innov	ekf2_innovations	Global Position
ekf2_innovations.output_tracking_error[0]	ekf2_innovations	Shared
ekf2_innovations.output_tracking_error[1]	ekf2_innovations	Shared
ekf2_innovations.output_tracking_error[2]	ekf2_innovations	Shared
ekf2_innovations.vel_pos_innov[3]	ekf2_innovations	Global Position
ekf2_innovations.vel_pos_innov[4]	ekf2_innovations	Global Position
ekf2_innovations.vel_pos_innov[5]	ekf2_innovations	Altitude
estimator_status.hagl_test_ratio	estimator_status	Altitude
estimator_status.hgt_test_ratio	estimator_status	Altitude
estimator_status.mag_test_ratio	estimator_status	Shared
estimator_status.pos_horiz_accuracy	estimator_status	Global Position
estimator_status.pos_test_ratio	estimator_status	Global Position
estimator_status.pos_vert_accuracy	estimator_status	Altitude
estimator_status.vibe[0]	estimator_status	Mechanical/Electrical
estimator_status.vibe[1]	estimator_status	Mechanical/Electrical
estimator_status.vibe[2]	estimator_status	Mechanical/Electrical
manual_control_setpoint.r	manual_control_setpoint	Shared
manual_control_setpoint.x	manual_control_setpoint	Shared
manual_control_setpoint.y	manual_control_setpoint	Shared
manual_control_setpoint.z	manual_control_setpoint	Shared

Continued on next page

Table A1 – continued from previous page

Telemetry channel	PX4 topic	Declared subsystem
rate_ctrl_status.pitchspeed_integ	rate_ctrl_status	Mechanical/Electrical
rate_ctrl_status.rollspeed_integ	rate_ctrl_status	Mechanical/Electrical
rate_ctrl_status.yawspeed_integ	rate_ctrl_status	Mechanical/Electrical
sensor_combined.accelerometer_m_s2[0]	sensor_combined	Mechanical/Electrical
sensor_combined.accelerometer_m_s2[1]	sensor_combined	Mechanical/Electrical
sensor_combined.accelerometer_m_s2[2]	sensor_combined	Mechanical/Electrical
sensor_combined.gyro_rad[0]	sensor_combined	Mechanical/Electrical
sensor_combined.gyro_rad[1]	sensor_combined	Mechanical/Electrical
sensor_combined.gyro_rad[2]	sensor_combined	Mechanical/Electrical
sensor_preflight.accel_inconsistency_m_s_s	sensor_preflight	Mechanical/Electrical
sensor_preflight.gyro_inconsistency_rad_s	sensor_preflight	Mechanical/Electrical
vehicle_attitude.pitchspeed	vehicle_attitude	Mechanical/Electrical
vehicle_attitude.q[0]	vehicle_attitude	Mechanical/Electrical
vehicle_attitude.q[1]	vehicle_attitude	Mechanical/Electrical
vehicle_attitude.q[2]	vehicle_attitude	Mechanical/Electrical
vehicle_attitude.q[3]	vehicle_attitude	Mechanical/Electrical
vehicle_attitude.rollspeed	vehicle_attitude	Mechanical/Electrical
vehicle_attitude.yawspeed	vehicle_attitude	Mechanical/Electrical
vehicle_attitude_setpoint.pitch_body	vehicle_attitude_setpoint	Mechanical/Electrical
vehicle_attitude_setpoint.roll_body	vehicle_attitude_setpoint	Mechanical/Electrical
vehicle_attitude_setpoint.yaw_body	vehicle_attitude_setpoint	Mechanical/Electrical
vehicle_land_detected.ground_contact	vehicle_land_detected	Altitude
vehicle_land_detected.landed	vehicle_land_detected	Altitude
vehicle_land_detected.maybe_landed	vehicle_land_detected	Altitude
vehicle_local_position.ax	vehicle_local_position	Global Position
vehicle_local_position.ay	vehicle_local_position	Global Position
vehicle_local_position.az	vehicle_local_position	Altitude
vehicle_local_position.dist_bottom	vehicle_local_position	Altitude
vehicle_local_position.dist_bottom_rate	vehicle_local_position	Altitude
vehicle_local_position.eph	vehicle_local_position	Global Position
vehicle_local_position.epv	vehicle_local_position	Altitude
vehicle_local_position.vx	vehicle_local_position	Global Position
vehicle_local_position.vy	vehicle_local_position	Global Position
vehicle_local_position.vz	vehicle_local_position	Altitude
vehicle_local_position.x	vehicle_local_position	Global Position
vehicle_local_position.y	vehicle_local_position	Global Position
vehicle_local_position.z	vehicle_local_position	Altitude
vehicle_rates_setpoint.pitch	vehicle_rates_setpoint	Mechanical/Electrical
vehicle_rates_setpoint.roll	vehicle_rates_setpoint	Mechanical/Electrical
vehicle_rates_setpoint.yaw	vehicle_rates_setpoint	Mechanical/Electrical
vehicle_status.arming_state	vehicle_status	Shared
vehicle_status.nav_state	vehicle_status	Shared
vehicle_vision_position.x	vehicle_vision_position	External Position
vehicle_vision_position.y	vehicle_vision_position	External Position
vehicle_vision_position.z	vehicle_vision_position	External Position

Appendix B. Reproducibility and Statistical Details

Appendix B.1. Software and Frozen Versions

The limited recomputation used Python 3.12.13, NumPy 2.3.5, pandas 3.0.1, scikit-learn 1.8.0, LightGBM 4.6.0, joblib 1.5.2, PyYAML 6.0.3, and pyulog 1.2.4. TreeSHAP contributions were obtained from LightGBM’s `pred_contrib` interface. No model was retrained or tuned. Model seeds were 0–4. The PX4 source-tree scan and controlled replay used commit 82aa24adfca29321cfd1209e287eab6c2b16780e. The CATCH feasibility execution used the official core at commit 3647c69be5eb56649b072596cf89098e689e20c3.

Appendix B.2. Bootstrap and Split Settings

Absolute real-flight confidence intervals used 1000 complete-flight cluster-bootstrap repetitions with random seed 20260825. Each bootstrap sample was evaluated for all five frozen model seeds and the replicate metric was their mean. ALFA used the same repetition count and seed with its 46 sequences as clusters; Uncategorized screening used its 1017 flights. Controlled-replay intervals used 5000 crossed cluster resamples of three source logs and four mutation types. The fixed chronological split contained 152,340/32,138/34,059 train/validation/test windows; the 14-window purged split con-

tained 133,353/8,536/17,966; leave-log-out contained 970/206/213 train/validation/test logs and 149,774/37,324/31,439 windows. All window models used length 96, stride 8, prediction horizon 12, 87 channels, and 1566 derived features.

Appendix B.3. Conservative Source-Path Exclusion

The confirmed firmware metadata indicated PX4FMU_V2 with NuttX. The conservative sensitivity therefore excluded edges whose normalized source path contained markers for examples, templates, tests, POSIX, QURT, Bebop, Linux, Snapdragon, simulator, or replay code. The rule removed 35 of 180 edges: 14 for examples, 8 for POSIX, 7 for simulator/replay, 2 for QURT, and one each for Linux, Snapdragon, tests, and templates. The remaining 145 edges covered 36 modules and all 18 evaluated topics. The filtered table is a strict subset of the original table and passed the same evidence-conservation calculation. This path-based sensitivity is not equivalent to resolving CMake configuration, board manifests, conditional compilation, linking, or runtime activation for an actual PX4FMU_V2 build.

References

- Keipour, A.; Mousaei, M.; Scherer, S. Automatic Real-Time Anomaly Detection for Autonomous Aerial Vehicles. In Proceedings of the 2019 International Conference on Robotics and Automation (ICRA), Montreal, QC, Canada, 20–24 May 2019; pp. 5679–5685. <https://doi.org/10.1109/ICRA.2019.8794286>.
- Wei, J.; Li, H.; Li, Y.; Zhao, Y.; Yao, D.; Lu, B.; Ye, X.; Im, S.-K.; Wang, Y.; Yang, X. AeroTSBoost: Temporal-Statistical Boosting for Real-World UAV Telemetry Anomaly Mining. *arXiv* **2026**, arXiv:2605.25639. <https://arxiv.org/abs/2605.25639>.
- Fourlas, G.K.; Karras, G.C. A Survey on Fault Diagnosis and Fault-Tolerant Control Methods for Unmanned Aerial Vehicles. *Machines* **2021**, *9*, 197. <https://doi.org/10.3390/machines9090197>.
- Puchalski, R.; Giernacki, W. UAV Fault Detection Methods, State-of-the-Art. *Drones* **2022**, *6*, 330. <https://doi.org/10.3390/drones6110330>.
- Tan, I.W.H.; Poskitt, C.M.; Jiang, L.; Shar, L.K. Anomaly Management in Unmanned Aerial Vehicles: A Systematic Literature Review. *IEEE Access* **2026**, *14*, 117991–118010. <https://doi.org/10.1109/ACCESS.2026.3716292>.
- Kabaoglu, A.; Sariel, S. UAV-SEAD: State Estimation Anomaly Dataset for UAVs. *arXiv* **2026**, arXiv:2602.13900. <https://arxiv.org/abs/2602.13900>.
- Meier, L.; Honegger, D.; Pollefeys, M. PX4: A Node-Based Multithreaded Open Source Robotics Framework for Deeply Embedded Platforms. In Proceedings of the 2015 IEEE International Conference on Robotics and Automation (ICRA), Seattle, WA, USA, 26–30 May 2015; pp. 6235–6240. <https://doi.org/10.1109/ICRA.2015.7140074>.
- PX4 Development Team. uORB Messaging and Publication/Subscription Graph. Available online: <https://docs.px4.io/main/en/middleware/uorb> (accessed on 25 August 2026).
- Lundberg, S.M.; Lee, S.-I. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems*; Curran Associates, Inc.: Red Hook, NY, USA, 2017; Volume 30.
- Lundberg, S.M.; Erion, G.; Chen, H.; DeGrave, A.; Prutkin, J.M.; Nair, B.; Katz, R.; Himmelfarb, J.; Bansal, N.; Lee, S.-I. From Local Explanations to Global Understanding with Explainable AI for Trees. *Nat. Mach. Intell.* **2020**, *2*, 56–67. <https://doi.org/10.1038/s42256-019-0138-9>.
- Jones, J.A.; Harrold, M.J.; Stasko, J.T. Visualization of Test Information to Assist Fault Localization. In Proceedings of the 24th International Conference on Software Engineering (ICSE 2002), Orlando, FL, USA, 19–25 May 2002; pp. 467–477. <https://doi.org/10.1145/581339.581397>.
- Wong, W.E.; Gao, R.; Li, Y.; Abreu, R.; Wotawa, F. A Survey on Software Fault Localization. *IEEE Trans. Softw. Eng.* **2016**, *42*, 707–740. <https://doi.org/10.1109/TSE.2016.2521368>.
- Pearson, S.; Campos, J.; Just, R.; Fraser, G.; Abreu, R.; Ernst, M.D.; Pang, D.; Keller, B. Evaluating and Improving Fault Localization. In Proceedings of the 2017 IEEE/ACM 39th International

- Conference on Software Engineering (ICSE), Buenos Aires, Argentina, 20–28 May 2017; pp. 609–620. <https://doi.org/10.1109/ICSE.2017.62>.
14. Papadakis, M.; Kintis, M.; Zhang, J.; Jia, Y.; Le Traon, Y.; Harman, M. Mutation Testing Advances: An Analysis and Survey. *Adv. Comput.* **2019**, *112*, 275–378. <https://doi.org/10.1016/bs.adcom.2018.03.015>.
 15. Adaika, Z.; Al-Haddad, L.A.; Giernacki, W.; Jaber, A.A.; Boumehraz, M.; Hamzah, M.N.; Flayyih, M.A. Fault Detection and Diagnosis Methodologies for Unmanned Aerial Vehicles: State-of-the-Art. *J. Intell. Robot. Syst.* **2025**, *111*, 63. <https://doi.org/10.1007/s10846-025-02267-8>.
 16. Keipour, A.; Mousaei, M.; Scherer, S. ALFA: A Dataset for UAV Fault and Anomaly Detection. *Int. J. Robot. Res.* **2021**, *40*, 515–520. <https://doi.org/10.1177/0278364920966642>.
 17. Liu, F.T.; Ting, K.M.; Zhou, Z.-H. Isolation Forest. In Proceedings of the 2008 Eighth IEEE International Conference on Data Mining, Pisa, Italy, 15–19 December 2008; pp. 413–422. <https://doi.org/10.1109/ICDM.2008.17>.
 18. Zhang, C.; Song, D.; Chen, Y.; Feng, X.; Lumezanu, C.; Cheng, W.; Ni, J.; Zong, B.; Chen, H.; Chawla, N.V. A Deep Neural Network for Unsupervised Anomaly Detection and Diagnosis in Multivariate Time Series Data. *Proc. AAAI Conf. Artif. Intell.* **2019**, *33*, 1409–1416. <https://doi.org/10.1609/aaai.v33i01.33011409>.
 19. Su, Y.; Zhao, Y.; Niu, C.; Liu, R.; Sun, W.; Pei, D. Robust Anomaly Detection for Multivariate Time Series through Stochastic Recurrent Neural Network. In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, Anchorage, AK, USA, 4–8 August 2019; pp. 2828–2837. <https://doi.org/10.1145/3292500.3330672>.
 20. Deng, A.; Hooi, B. Graph Neural Network-Based Anomaly Detection in Multivariate Time Series. *Proc. AAAI Conf. Artif. Intell.* **2021**, *35*, 4027–4035. <https://doi.org/10.1609/aaai.v35i5.16523>.
 21. Tuli, S.; Casale, G.; Jennings, N.R. TranAD: Deep Transformer Networks for Anomaly Detection in Multivariate Time Series Data. *Proc. VLDB Endow.* **2022**, *15*, 1201–1214. <https://doi.org/10.14778/3514061.3514067>.
 22. Xu, J.; Wu, H.; Wang, J.; Long, M. Anomaly Transformer: Time Series Anomaly Detection with Association Discrepancy. In Proceedings of the International Conference on Learning Representations, Virtual, 25–29 April 2022. Available online: https://openreview.net/forum?id=LzQQ89U1qm_ (accessed on 25 August 2026).
 23. Wu, X.; Qiu, X.; Li, Z.; Wang, Y.; Hu, J.; Guo, C.; Xiong, H.; Yang, B. CATCH: Channel-Aware Multivariate Time Series Anomaly Detection via Frequency Patching. In Proceedings of the International Conference on Learning Representations, Singapore, 24–28 April 2025. Available online: <https://openreview.net/forum?id=m08aK3xxdJ> (accessed on 25 August 2026).
 24. Liu, Z.; Gao, M.; Jiao, P. GCAD: Anomaly Detection in Multivariate Time Series from the Perspective of Granger Causality. *Proc. AAAI Conf. Artif. Intell.* **2025**, *39*, 19041–19049. <https://doi.org/10.1609/aaai.v39i18.34096>.
 25. Ke, G.; Meng, Q.; Finley, T.; Wang, T.; Chen, W.; Ma, W.; Ye, Q.; Liu, T.-Y. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems*; Curran Associates, Inc.: Red Hook, NY, USA, 2017; Volume 30.
 26. Davis, J.; Goadrich, M. The Relationship between Precision-Recall and ROC Curves. In Proceedings of the 23rd International Conference on Machine Learning, Pittsburgh, PA, USA, 25–29 June 2006; pp. 233–240. <https://doi.org/10.1145/1143844.1143874>.
 27. Saito, T.; Rehmsmeier, M. The Precision–Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLoS ONE* **2015**, *10*, e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
 28. Tatbul, N.; Lee, T.J.; Zdonik, S.; Alam, M.; Gottschlich, J. Precision and Recall for Time Series. In *Advances in Neural Information Processing Systems*; Curran Associates, Inc.: Red Hook, NY, USA, 2018; Volume 31.
 29. Wenig, P.; Schmidl, S.; Papenbrock, T. TimeEval: A Benchmarking Toolkit for Time Series Anomaly Detection Algorithms. *Proc. VLDB Endow.* **2022**, *15*, 3678–3681. <https://doi.org/10.14778/3554821.3554873>.
 30. Wu, R.; Keogh, E.J. Current Time Series Anomaly Detection Benchmarks Are Flawed and Are Creating the Illusion of Progress. *IEEE Trans. Knowl. Data Eng.* **2023**, *35*, 2421–2429. <https://doi.org/10.1109/TKDE.2021.3112126>.

31. Ribeiro, M.T.; Singh, S.; Guestrin, C. Why Should I Trust You?: Explaining the Predictions of Any Classifier. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, 13–17 August 2016; pp. 1135–1144. <https://doi.org/10.1145/2939672.2939778>.
1031
1032
1033
1034
32. Sundararajan, M.; Taly, A.; Yan, Q. Axiomatic Attribution for Deep Networks. In Proceedings of the 34th International Conference on Machine Learning, Sydney, Australia, 6–11 August 2017; Volume 70, pp. 3319–3328.
1035
1036
1037
33. Bento, J.; Saleiro, P.; Cruz, A.F.; Figueiredo, M.A.T.; Bizarro, P. TimeSHAP: Explaining Recurrent Models through Sequence Perturbations. In Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, Virtual, 14–18 August 2021; pp. 2565–2573. <https://doi.org/10.1145/3447548.3467166>.
1038
1039
1040
1041
34. Adebayo, J.; Gilmer, J.; Muelly, M.; Goodfellow, I.; Hardt, M.; Kim, B. Sanity Checks for Saliency Maps. In *Advances in Neural Information Processing Systems*; Curran Associates, Inc.: Red Hook, NY, USA, 2018; Volume 31.
1042
1043
1044
35. Hooker, S.; Erhan, D.; Kindermans, P.-J.; Kim, B. A Benchmark for Interpretability Methods in Deep Neural Networks. In *Advances in Neural Information Processing Systems*; Curran Associates, Inc.: Red Hook, NY, USA, 2019; Volume 32.
1045
1046
1047
36. Hedström, A.; Weber, L.; Krakowczyk, D.; Bareeva, D.; Motzkus, F.; Samek, W.; Lapuschkin, S.; Höhne, M.M.-C. Quantus: An Explainable AI Toolkit for Responsible Evaluation of Neural Network Explanations and Beyond. *J. Mach. Learn. Res.* **2023**, *24*, 1–11.
1048
1049
1050